L15 COMBINING MULTIPLE LEARNERS

Outline

- COMBINING MULTIPLE LEARNERS:
- Rationale, Generating Diverse Learners, Voting, Error-Correcting Output Codes, Bagging, Boosting.
- Chapter 17.1, 17.2, 17.4 - 17.7
- TextBook 2: Ethem Alpaydin, *Introduction to Machine Learning*, 2nd edition, MIT Press. 2010.

Combining Multiple Learners

- We discussed many different learning algorithms
- Though these are generally successful, no one single algorithm is always the most accurate.
- We need models composed of multiple learners that complement each other so that by combining them, we attain higher accuracy.

Rationale

- Machine learning models are not like traditional software solutions
- These models need constant updates as new data becomes available for accurate and reliable predictions
- In complex and sensitive scenarios, relying on a single model may not be sufficient to generate an optimal result
- Ensemble (a group taken together as a whole) modeling can help improve overall model performance
- Think of ensemble models as an orchestra of musicians, where each person specializes in a specific instrument like piano, trumpet, drum, and more.
- The combination of those skills creates a harmonious melody

General Terms

- Model: describes the output of the algorithm that is trained with data. This model is then used for making predictions.
- Algorithm: can be any machine learning algorithm such as logistic regression, decision tree, etc
- Base model: Models when used as inputs of ensemble methods, are called ”base models,” and the end result is an ensemble model.
- Ensemble method: a machine learning technique that combines several base models in order to produce one optimal predictive model
- Base learners: are the first level of an ensemble learning architecture, and each one of them is trained to make individual predictions.
- Meta learners: are in the second level, and they are trained on the output of the base learners
- Weak learners: The individual models that we combine are also known as weak learners (base learners) because they either have a high bias or high variance. Because they either have high bias or variance, weak learners cannot learn efficiently and perform poorly

Score method

- model.score() : for classification or regression problems, Scores are between 0 and 1, with a larger score indicating a better fit.
- Score = correct predictions / total number of data points

Ensemble Techniques

- Simple Ensemble Techniques
  - Max Voting
  - Averaging
  - Weighted Averaging
- Advanced Ensemble techniques
  - Bagging
  - Boosting
  - Stacking
  - Blending

Voting

- It is called voting because like in election voting, the premise is that ‘the majority rules’.
- The max voting method is generally used for classification problems.
  - In this technique, multiple models are used to make predictions for each data point.
  - The predictions by each model are considered as a ‘vote’.
  - The predictions which we get from the majority of the models are used as the final prediction.
- Ex1: Voting to rate a movie among 5 colleagues (out of 5 rating);
  - If three of them rated as 4 while two of them gave it a 5. The majority gave a rating of 4, the final rating will be taken as 4.

Colleague 1	Colleague 2	Colleague 3	Colleague 4	Colleague 5	Final rating
5	4	5	4	4	4

- Ex2: We have three classifiers with the following predictions:
  - classifier 1 – class A, classifier 2 – class B, classifier 3 – class B
  - The final prediction here would be class B since it has the most votes

Voting – Manual Implementation

```
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
import statistics as st
import warnings
warnings.filterwarnings('ignore’)
# SPLITTING THE DATASET
df = pd.read_csv('heart.csv’)
x = df.drop('target', axis = 1)
y = df['target’]
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)
```

Voting - Implementation

```
# MODELS CREATION
model1 = DecisionTreeClassifier()
model2 = KNeighborsClassifier()
model3= LogisticRegression()
model1.fit(x_train,y_train)
model2.fit(x_train,y_train)
model3.fit(x_train,y_train)
# PREDICTION
pred1=model1.predict(x_test)
pred2=model2.predict(x_test)
pred3=model3.predict(x_test)
# FINAL_PREDICTION
final_pred = np.array([])
for i in range(0,len(x_test)):
    final_pred = np.append(final_pred, st.mode([pred1[i], pred2[i], pred3[i]]))
print(final_pred)
```

Voting – Scikit learn Implementation

```
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
import statistics as st
import warnings
warnings.filterwarnings('ignore')
from sklearn.ensemble import VotingClassifier
model1 = LogisticRegression(random_state=1)
model2 = DecisionTreeClassifier(random_state=1)
model = VotingClassifier(estimators=[('lr', model1), ('dt', model2)], voting='hard')
model.fit(x_train,y_train)
model.score(x_test,y_test)
```

Bagging

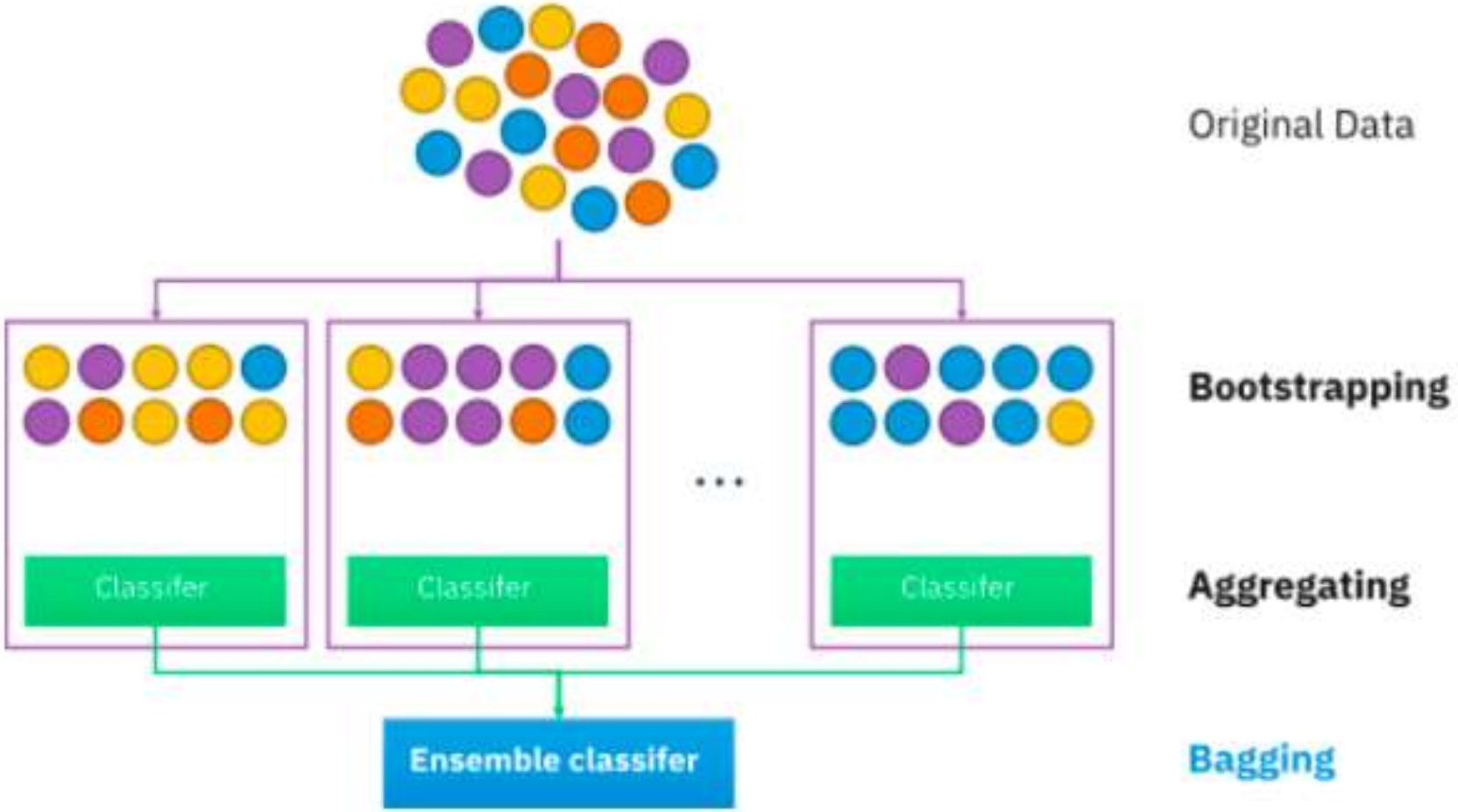
- BAGGing= Bootstrapping and Aggregation
- BAGGing gets its name because it combines Bootstrapping and Aggregation to form one ensemble model.

Bagging

- The idea behind bagging
- Combining the results of multiple models (for instance, all decision trees) to get a generalized result.
- If we create all the models on the same set of data and combine it, will it be useful?
- There is a high chance that these models will give the same result since they are getting the same input.
- So how can we solve this problem? One of the techniques is bootstrapping
- Bootstrapping is a sampling technique in which we create subsets of observations from the original dataset, with replacement.
- Bagging (or Bootstrap Aggregating) technique uses these subsets (bags) to get a fair idea of the distribution (complete set).
- The size of subsets created for bagging may be less than the original set.
- Bagging uses parallel operations

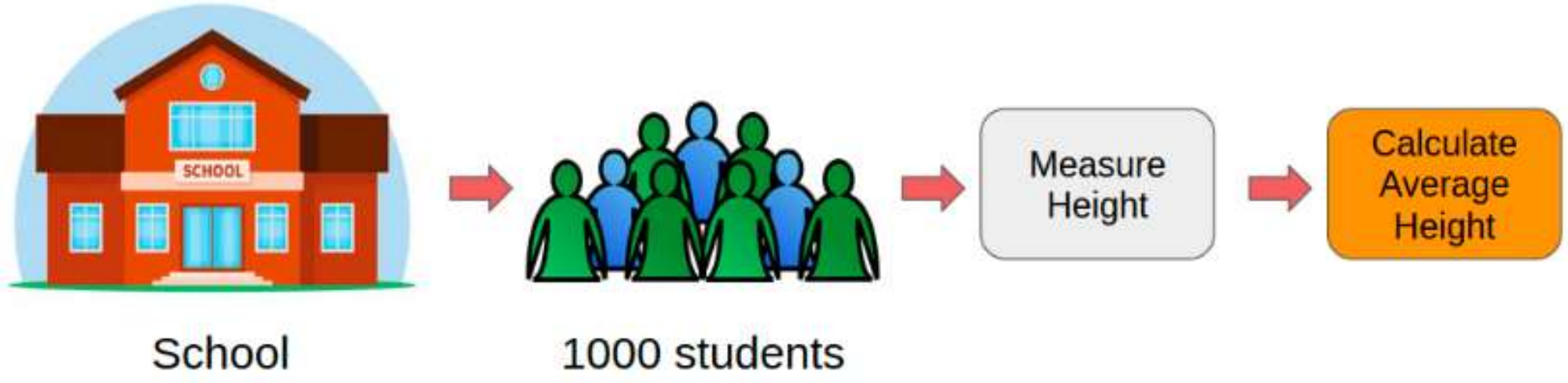
Bootstrap Sampling

- Sampling: With respect to statistics, sampling is the process of selecting a subset of items from a vast collection of items (population) to estimate a certain characteristic of the entire population
- Sampling with replacement: It means a data point in a drawn sample can reappear in future drawn samples as well



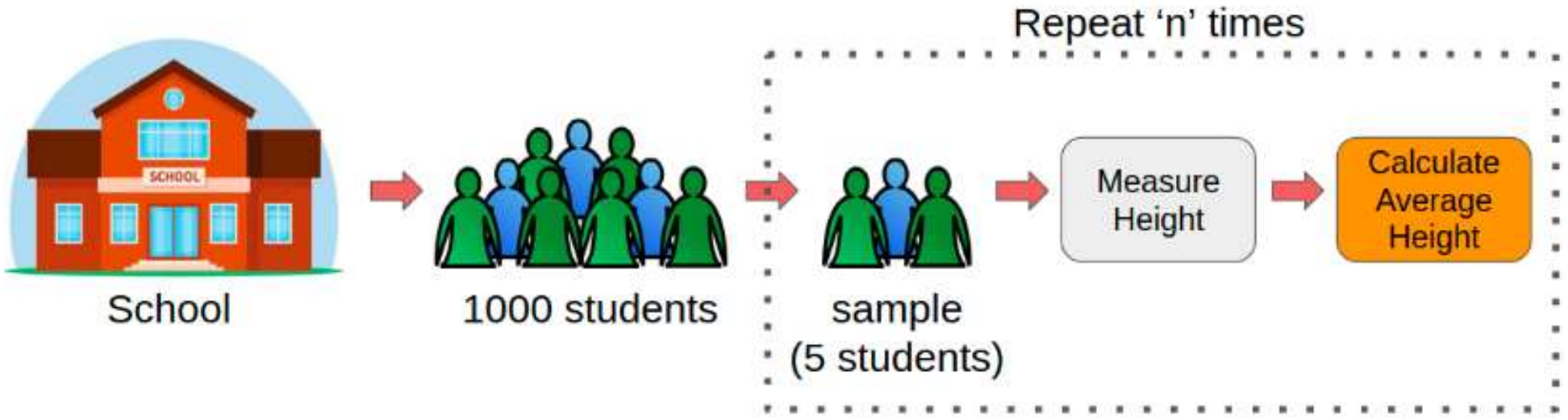
Why Do We Need Bootstrap Sampling?

- To find the mean height of all the students in a school (which has a total population of 1,000).
- How can we perform this task?
- One approach is to measure the height of all the students and then compute the mean height
- However, this would be a tedious task to individually measure the heights of 1,000 students and then compute the mean height.
- A smarter approach is Bootstrap Sampling



Why Do We Need Bootstrap Sampling?

- Instead of measuring the heights of all the students, we can draw a random sample of 5 students and measure their heights.
- We would repeat this process 20 times and then average the collected height data of 100 students (5 x 20).
- This average height would be an estimate of the mean height of all the students of the school.



Why Do We Need Bootstrap Sampling?

- Estimate the population mean with the help of bootstrap sampling
- Create a Gaussian distribution (population) of 10,000 elements with the population mean being 500:
- Next, draw 40 samples of size 5 from the distribution (population) and compute the mean for every sample:

Built-in library functions used:

1. random.normal(loc=0.0, scale=1.0, size=None): Draw random samples from a normal (Gaussian) distribution.

- The probability density function of the normal distribution, derived by Gauss and Laplace is often called the bell curve because of its characteristic shape
- loc -Mean (“centre”) of the distribution.
- Scale - Standard deviation (spread or “width”) of the distribution. Must be non-negative.
- Size - int If the given shape is (n), then n samples are drawn.

2. random.sample(seq, n): random sampling without replacement Generate n unique samples (multiple items) from a sequence without repetition. Here, A seq can be a list, set, string, tuple. Sample without replacement

3. numpy.ndarray.tolist(): With the help of numpy.ndarray.tolist() method, we can have the list of data elements which is converted from an ndarray using ndarray.tolist() method

- For a 1D array, a.tolist() is almost the same as list(a), except that tolist changes numpy scalars to Python scalars

Sampling vs Bootstrap Sampling

```
import numpy as np

import seaborn as sns

import random

import matplotlib.pyplot as plt

# normal distribution

x = np.random.normal(loc= 500.0, scale=1.0, size=10000)

print("mean=", np.mean(x))

plt.hist(x, bins=100)

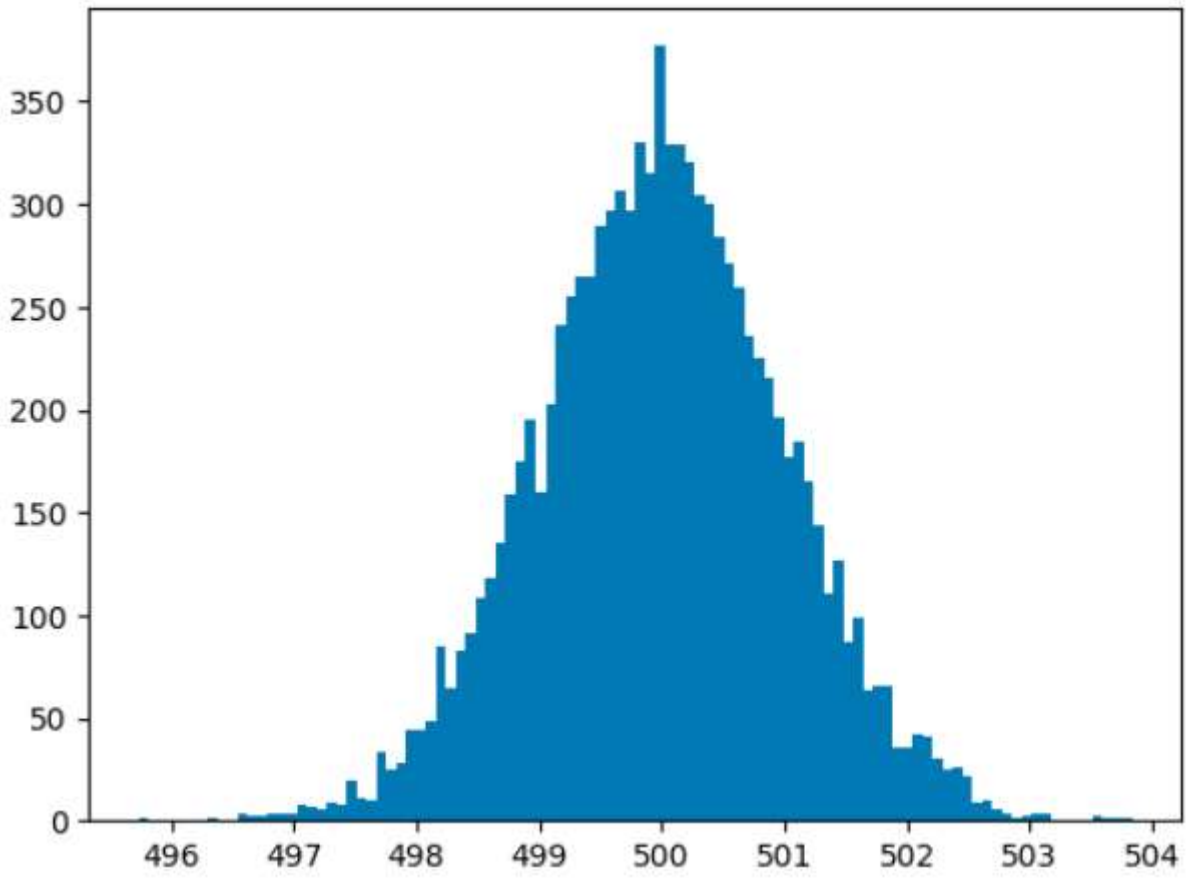
#plt.hist(x)

plt.show()
```

```
# Bootstrap Sampling
sample_mean = []
for i in range(40):
        y = random.sample(x.tolist(), 5)
        print("y=", y)
        avg = np.mean(y)
        sample_mean.append(avg)
print("Display mean=", sample_mean)
print("sample mean", np.mean(sample_mean))
```

mean= 499.99405099816175
sample mean 499.9441358560316
The average of the mean values of all the 40 samples is close to the population mean

Sampling vs Bootstrap Sampling



Bootstrap Sampling

Benefits of bootstrapping:

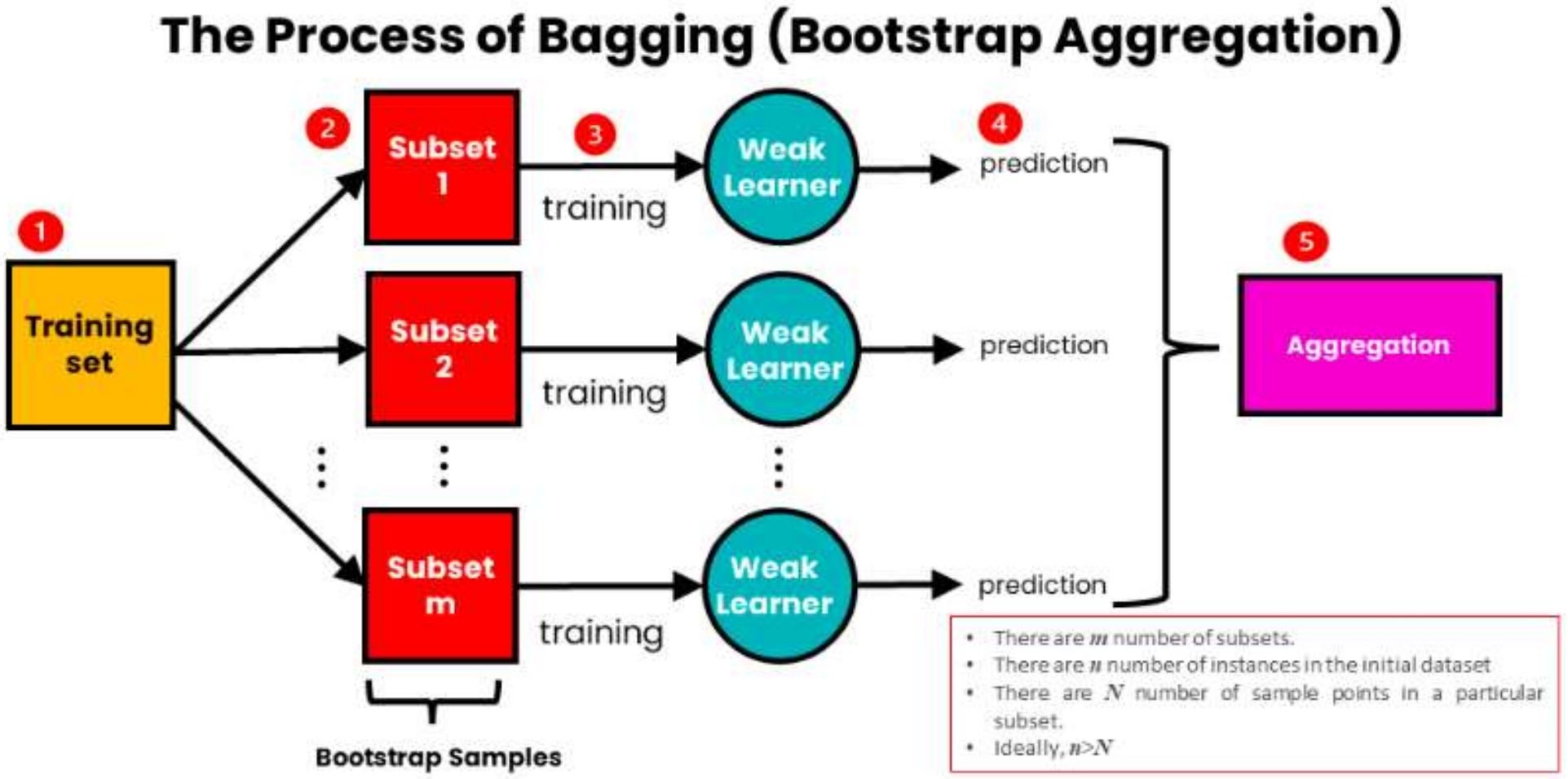
- When we have to estimate a parameter of a large population, we can take the help of Bootstrap Sampling
- The estimated parameter by bootstrap sampling is comparable to the actual population parameter
- Since we only need a few samples for bootstrapping, the computation requirement is very less

Bootstrap sampling

- In bootstrap sampling, some original examples appear more than once and some original examples are not present in the sample.
- To create a sub-dataset with n elements, select a random element from the original dataset n times.
- Bootstrap sampling Python code: Create m datasets where the number of elements in each dataset is n.

```
def bootstrap_sample(original_dataset, n):
    sub_dataset = []
    for i in range(n):
        sub_dataset.append(random_one_element(original_dataset))
    return sub_dataset
for i in range(m):
        sub_dataset = bootstrap_sample(train_dataset, n)
```

Bagging

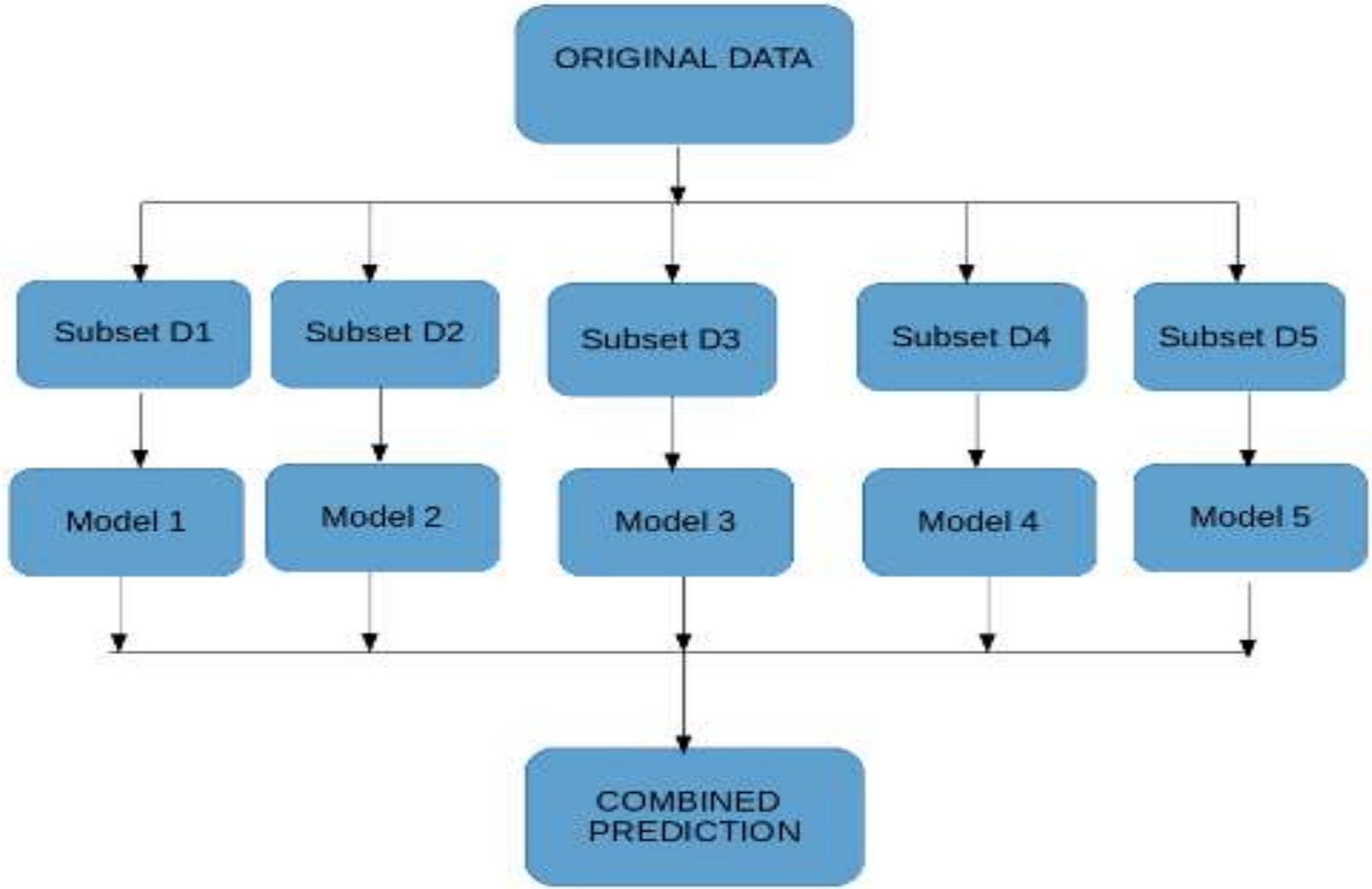

The Process of Bagging (Bootstrap Aggregation)
1
Training set
2
Subset 1
Subset 2
Subset m
3
training
training
training
Weak Learner
Weak Learner
Weak Learner
4
prediction
prediction
prediction
5
Aggregation
Bootstrap Samples
There are m number of subsets.
There are n number of instances in the initial dataset
There are N number of sample points in a particular subset.
Ideally, n>N

Bagging

The steps of bagging:

1. We have an initial training dataset containing n-number of instances.
2. We create a m-number of subsets of data from the training set. We take a subset of N sample points from the initial dataset for each subset. Each subset is taken with replacement. This means that a specific data point can be sampled more than once.
3. For each subset of data, we train the corresponding weak learners independently. These models are homogeneous, meaning that they are of the same type.
4. Each model makes a prediction.
5. The predictions are aggregated into a single prediction. For this, either max voting or averaging is used.

Bagging


ORIGINAL DATA
Subset D1
Subset D2
Subset D3
Subset D4
Subset D5
Model 1
Model 2
Model 3
Model 4
Model 5
COMBINED PREDICTION

Bagging Implementation

- Scikit-learn uses a `BaggingClassifier` and a `BaggingRegressor`
- The bagging meta-estimator fits each base model on random subsets of the original dataset.
- It then computes the final prediction by aggregating individual base model predictions.
- Aggregation is done by voting or averaging.
- The method reduces the variance of estimators by introducing randomization in their construction process.

Bagging Implementation - Bagging classifier

Import the `BaggingClassifier`,

Import a base estimator – a decision tree classifier,

Create an instance of the `BaggingClassifier`

The bagging classifier takes arguments:

- base estimator – here, a decision tree classifier,
- n - number of estimators in the ensemble,
- `max_samples` to define the number of samples that will be drawn from the training set for each base estimator,
- `max_features` to dictate the number of features that will be used to train each base estimator.
- Fit this classifier on the training set and score:

```
bagging.fit(x_train, y_train)

bagging.score(x_test,y_test)
```

Bagging Implementation - Bagging classifier

```
import pandas as pd
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
df = pd.read_csv('heart.csv’)
x = df.drop('target', axis = 1)
y = df['target’]
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)
bagging = BaggingClassifier(base_estimator=DecisionTreeClassifier(), n_estimators=10, max_samples=0.5, max_features=0.5)
bagging.fit(x_train, y_train)
bagging.score(x_test,y_test)
```

Bagging Implementation – Bagging Regressor

- The same process for regression problems to work with regression estimators.

Bagging Implementation – Bagging Regressor

```
import pandas as pd
from sklearn.ensemble import BaggingRegressor
from sklearn.tree import DecisionTreeRegressor
df = pd.read_csv('heart.csv’)
x = df.drop('target', axis = 1)
y = df['target’]
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)
bagging = BaggingRegressor(estimator=DecisionTreeRegressor(), n_estimators=10, random_state=0)
bagging.fit(x_train, y_train)
bagging.score(x_test,y_test)
```

Boosting

- If a data point is incorrectly predicted by the first model, and then the next (probably all models), will combining the predictions provide better results?
- Such situations are handled by boosting
- Boosting is a sequential process, where each subsequent model attempts to correct the errors of the previous model.
- The succeeding models are dependent on the previous model.

Boosting

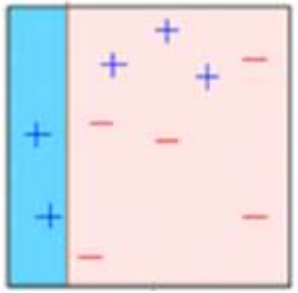
- The term “boosting” is used to describe a family of algorithms which are able to convert weak models to strong models.
- The model is weak if it has a substantial error rate
- Boosting incrementally builds an ensemble by training each model with the same dataset but where the weights of instances are adjusted according to the error of the last prediction.
- Unlike bagging, boosting is a sequential method

Boosting Steps – Pictorial illustration

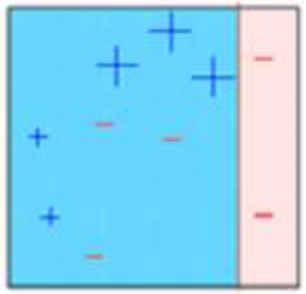
Boosting steps:

1. A subset is created from the original dataset.
2. Initially, all data points are given equal weights.
3. A base model is created on this subset.
4. This model is used to make predictions on the whole dataset
5. Errors are calculated using the actual values and predicted values.
6. The observations which are incorrectly predicted, are given higher weights. Here, the three misclassified blue-plus points will be given higher weights)
7. Another model is created and predictions are made on the dataset. This model tries to correct the errors from the previous model)
8. Similarly, multiple models are created, each correcting the errors of the previous model.
9. The final model (strong learner) is the weighted mean of all the models (weak learners)

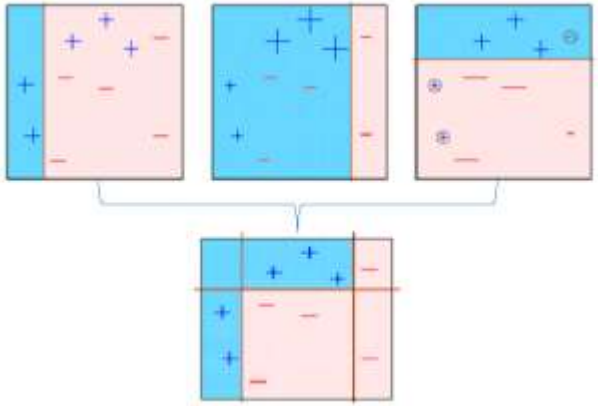
Boosting Steps – Pictorial illustration



1. A subset is created from the original dataset.
2. Initially, all data points are given equal weights.
3. A base model is created on this subset.
4. This model is used to make predictions on the whole dataset

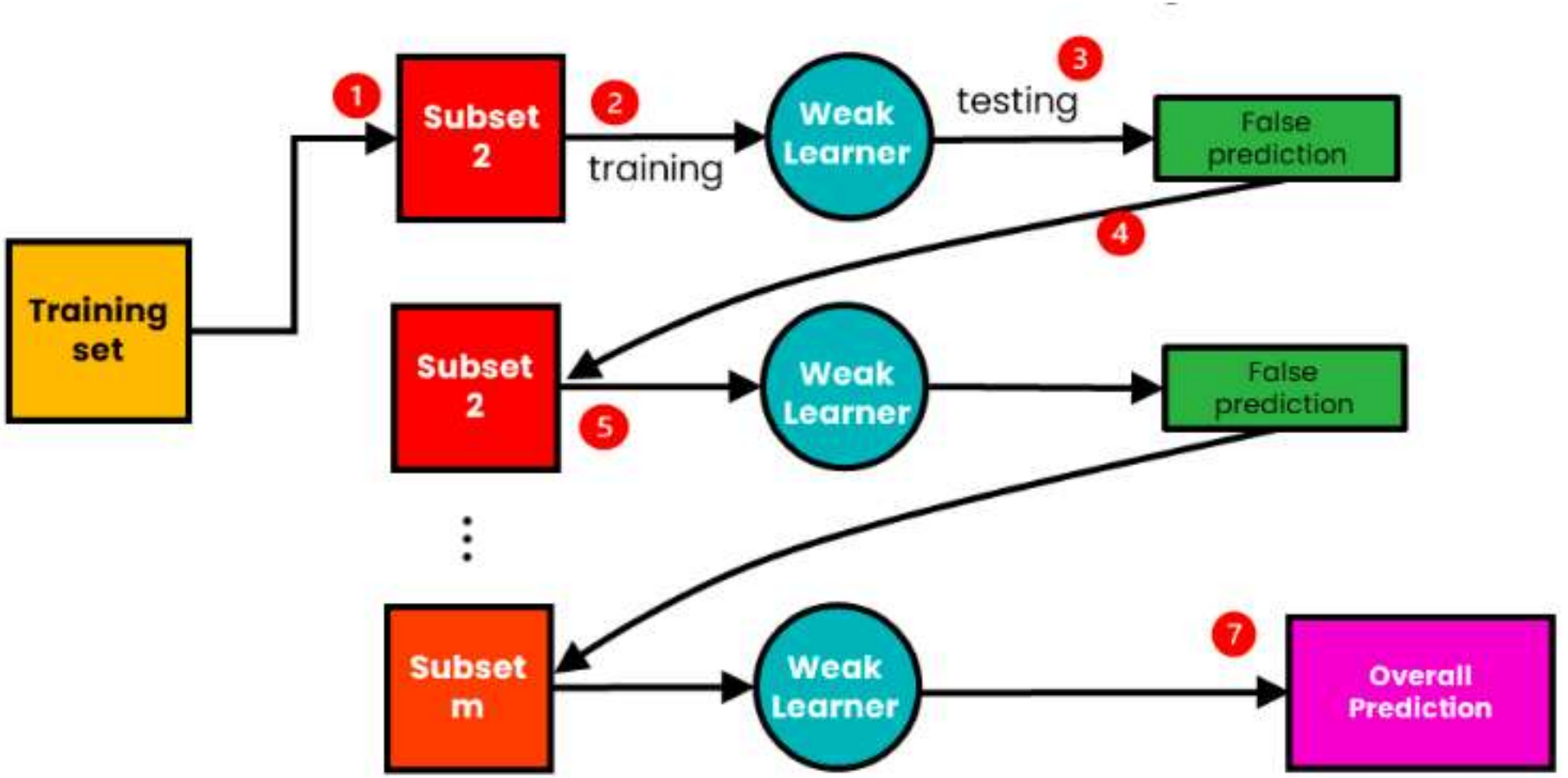


5. Errors are calculated using the actual values and predicted values.
6. The observations which are incorrectly predicted, are given higher weights.
(Here, the three misclassified blue-plus points will be given higher weights)
7. Another model is created and predictions are made on the dataset.
(This model tries to correct the errors from the previous model)



8. Similarly, multiple models are created, each correcting the errors of the previous model.
9. The final model (strong learner) is the weighted mean of all the models (weak learners).

Boosting


Training set
1
Subset 2
2
training
Weak Learner
3
testing
False prediction
4
Subset 2
5
Weak Learner
False prediction
Subset m
Weak Learner
7
Overall Prediction

Boosting

1. sample m-number of subsets from an initial training dataset.
2. Using the first subset, train the first weak learner.
3. test the trained weak learner using the training data. As a result of the testing, some data points will be incorrectly predicted.
4. Each data point with the wrong prediction is sent into the second subset of data, and this subset is updated.
5. Using this updated subset, train and test the second weak learner.
6. Continue with the following subset until the total number of subsets is reached.
7. We now have the total prediction. The overall prediction has already been aggregated at each step, so there is no need to calculate it.

Boosting – Implementation using sklearn AdaBoostClassifier

- AdaBoost, short for adaptive boosting, that uses the same training set over and over and thus need not be large
- AdaBoost can also combine an arbitrary number of base learners
- Once training is done, AdaBoost is a voting method.

Boosting – Implementation using sklearn AdaBoostClassifier

```
from sklearn.ensemble import AdaBoostClassifier
import pandas as pd
df = pd.read_csv('heart.csv’)
x = df.drop('target', axis = 1)
y = df['target’]
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)
model = AdaBoostClassifier(n_estimators=100)
model.fit(x_train, y_train)
model.score(x_test,y_test)
```

Algorithms based on Bagging and Boosting

- Bagging algorithms:
  - Bagging meta-estimator
  - Random forest
- Boosting algorithms:
  - AdaBoost
  - GBM
  - XGBM
  - Light GBM
  - CatBoost

Generating Diverse Learners

- There is no point in combining learners that always make similar diversity decisions
- The aim is to be able to find a set of diverse learners who differ in their decisions so that they complement each other.
- At the same time, there cannot be a gain in overall success unless the learners are accurate, at least in their domain of expertise.
- We therefore have this double task of maximizing individual accuracies and the diversity between learners.
- Different ways to achieve:

Generating Diverse Learners

Different ways to achieve:

1. Different Algorithms

- We can use different learning algorithms to train different base-learners.
- Different algorithms make different assumptions about the data and lead to different classifiers.
- Ex: one base-learner may be parametric and another may be nonparametric.

Background on Hyperparameters &Parameters

- Hyperparameters are said to be external to the model because the model cannot change its values during learning/training and will remain the same when training ends.
- Hyperparameters are parameters whose values control the learning process and determine the values of model parameters that a learning algorithm ends up learning.
- The prefix ‘hyper_’ suggests that they are ‘top-level’ parameters that control the learning process and the model parameters that result from it.

Hyperparameters

- Basically, anything in machine learning and deep learning that we decide their values or choose their configuration before training begins and whose values or configuration will remain the same when training ends is a hyperparameter
- Hyperparameters are used by the learning algorithm when it is learning but they are not part of the resulting model.
- At the end of the learning process, we have the trained model parameters
- The hyperparameters that were used during training are not part of this model.
- We cannot for instance know what hyperparameter values were used to train a model from the model itself, we only know the model parameters that were learned

Hyperparameters - Examples

- Train-test split ratio
- Learning rate in optimization algorithms (e.g. gradient descent)
- Choice of optimization algorithm (e.g., gradient descent, stochastic gradient descent, or Adam optimizer)
- Choice of activation function in a neural network (nn) layer (e.g. Sigmoid, ReLU, Tanh)
- The choice of cost or loss function the model will use
- Number of hidden layers in a nn
- Number of activation units in each layer
- The drop-out rate in nn (dropout probability)
- Number of iterations (epochs) in training a nn
- Number of clusters in a clustering task
- Kernel or filter size in convolutional layers
- Pooling size
- Batch size

Parameters

- Parameters are internal to the model.
- They are learned or estimated from the data during training as the algorithm tries to learn the mapping between the input features and the labels or targets
- parameters can change as it learns and these values are affected by the choice of hyperparameters
- So we set the hyperparameters before training begins and the learning algorithm uses them to learn the parameters.
- parameters are continuously being updated and the final ones at the end of the training constitute our model
- Model training typically starts with parameters being initialized to some values (random values or set to zeros). As training/learning progresses the initial values are updated using an optimization algorithm (e.g. gradient descent). The learning algorithm is continuously updating the parameter values as learning progress but hyperparameter values set by the model designer remain unchanged.

Parameters - Examples

- Examples of parameters
- The coefficients (or weights) of linear and logistic regression models.
- Weights and biases of a nn
- The cluster centroids in clustering

Setting hyperparameters

- Setting the right hyperparameter values is very important
- It directly impacts the performance of the model
- The process of choosing the best hyperparameters is called hyperparameter tuning

Generating Diverse Learners

Different ways to achieve:

2. Different Hyperparameters

- We can use the same learning algorithm but use it with different hyperparameters.
- Ex: number of hidden units in a multilayer perceptron, k in k-nearest neighbor, error threshold in decision trees, the kernel function in support vector machines, etc
- When we train multiple base-learners with different hyperparameter values, we average over this factor and reduce variance, and therefore error

Generating Diverse Learners

Different ways to achieve:

3. Different Input Representations

Separate base-learners may be using different representations of the same input object or event, making it possible to integrate different types of sensors/measurements/modalities.

- Different representations make different characteristics explicit allowing better identification.
- In many applications, there are multiple sources of information, and it is desirable to use all of these data to extract more information and achieve higher accuracy in prediction.

Generating Diverse Learners

Different ways to achieve:

4. Different Training Sets

- Another possibility is to train different base-learners by different subsets of the training set.
- This can be done randomly by drawing random training sets from the given sample; this is called bagging
- Or, the learners can be trained serially so that instances on which the preceding base-learners are not accurate are given more emphasis in training later base-learners;
- Ex: Boosting which actively try to generate complementary learners

Generating Diverse Learners

Different ways to achieve:

5. Diversity vs. Accuracy

- When we generate multiple base-learners, they need to be reasonably accurate but do not require them to be very accurate individually
- So they need not be, optimized separately for best accuracy.
- The base-learners are not chosen for their accuracy, but for their simplicity.
- However, baselearners need to be diverse, that is, accurate on different instances, specializing in subdomains of the problem.
- The final accuracy when the base-learners are combined is important rather than the accuracies of the individual base-learners.
- Ex: Assume we have a classifier that is 80 percent accurate.
- When we decide on a second classifier, consider only about how accurate it is on the 20 percent that the first classifier misclassifies

Error-Correcting Output Codes

- Error-correcting output codes (ECOC) (Dietterich and Bakiri 1995)
- The main classification task is defined in terms of a number of subtasks that are implemented by the base-learners.
- A set of simpler classification problems are defined, each specializing in one aspect of the task, and combining these simpler classifiers, we get the final classifier

Error-Correcting Output Codes

- Machine learning algorithms, like logistic regression and support vector machines, are designed for two-class (binary) classification problems.
- These algorithms must be modified for multi-class (more than two) classification problems
- The Error-Correcting Output Codes method is a technique that allows a multi-class classification problem to be reframed as multiple binary classification problems, allowing the use of native binary classification models to be used directly.

Error-Correcting Output Codes

- The approach is to prepare a binary encoding (e.g. a bitstring) to represent each class in the problem.
- Each bit in the string can be predicted by a separate binary classification problem.
- Each model receives the full input pattern and only predicts one position in the output string.
- During training, each model can be trained to produce the correct 0 or 1 output for the binary classification task.
- A prediction can then be made for new examples by using each model to make a prediction for the input to create the binary string, then compare the binary string to each class's known encoding.
- The class encoding that has the smallest distance to the prediction is then chosen as the output.

Error-Correcting Output Codes

- The key hyperparameter for the ECOC model is the encoding of class labels.
- This includes properties such as:
  - The choice of representation (bits, real numbers, etc.)
  - The encoding of each class label (random, etc.)
  - The length of representation (number of bits, etc.)
  - How predictions are mapped to classes (distance, etc.)
- Ex: if we wanted an encoding with bit strings with a length of 6 bits, and we had three classes, then we can specify the coding size as 2:
- encoding_length = code_size * num_classes = 2 * 3 = 6